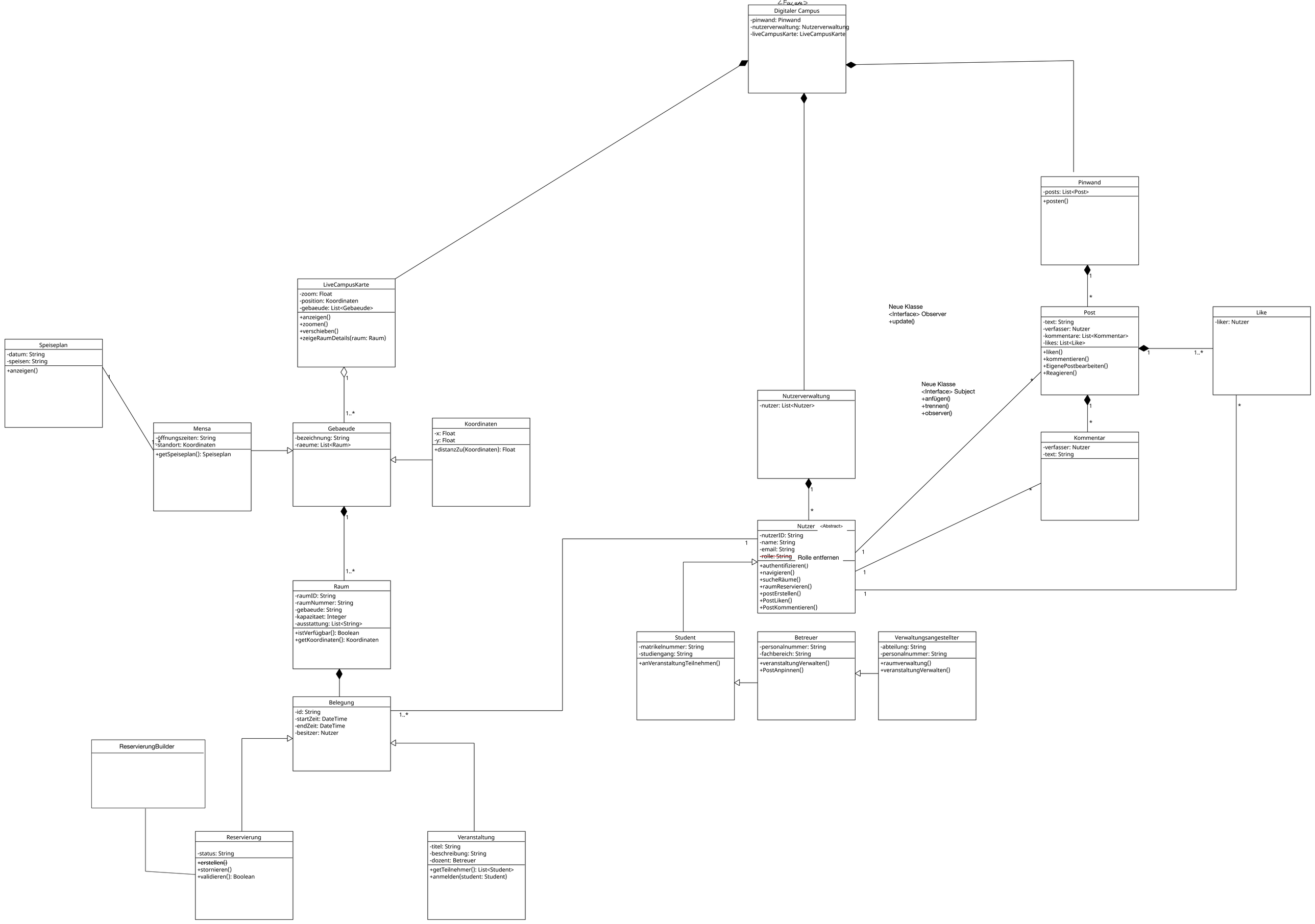
Digitaler Campus
-pinwand: Pinwand
-nutzerverwaltung: Nutzerverwaltung
-liveCampusKarte: LiveCampusKarte
LiveCampusKarte
-zoom: Float
-position: Koordinaten
-gebaeude: List<Gebaeude>
+anzeigen()
+zoomen()
+verschieben()
+zeigeRaumDetails(raum: Raum)
Pinwand
-posts: List<Post>
+posten()
Neue Klasse
<Interface> Observer
+update()
Post
-text: String
-verfasser: Nutzer
-kommentare: List<Kommentar>
-likes: List<Like>
+liken()
+kommentieren()
+EigenePostbearbeiten()
+Reagieren()
Like
-liker: Nutzer
Neue Klasse
<Interface> Subject
+anfügen()
+trennen()
+observer()
Kommentar
-verfasser: Nutzer
-text: String
Speiseplan
-datum: String
-speisen: String
+anzeigen()
Mensa
-öffnungszeiten: String
-standort: Koordinaten
+getSpeiseplan(): Speiseplan
Gebaeude
-bezeichnung: String
-raeume: List<Raum>
Koordinaten
-x: Float
-y: Float
+distanzZu(Koordinaten): Float
Nutzerverwaltung
-nutzer: List<Nutzer>
Nutzer
<Abstract>
-nutzerID: String
-name: String
-email: String
-rolle: String
Rolle entfernen
+authentifizieren()
+navigieren()
+sucheRäume()
+raumReservieren()
+postErstellen()
+PostLiken()
+PostKommentieren()
Raum
-raumID: String
-raumNummer: String
-gebaeude: String
-kapazitaet: Integer
-ausstattung: List<String>
+istVerfügbar(): Boolean
+getKoordinaten(): Koordinaten
Student
-matrikelnummer: String
-studiengang: String
+anVeranstaltungTeilnehmen()
Betreuer
-personalnummer: String
-fachbereich: String
+veranstaltungVerwalten()
+PostAnpinnen()
Verwaltungsangestellter
-abteilung: String
-personalnummer: String
+raumverwaltung()
+veranstaltungVerwalten()
Belegung
-id: String
-startZeit: DateTime
-endZeit: DateTime
-besitzer: Nutzer
ReservierungBuilder
Reservierung
-status: String
+erstellen()
+stornieren()
+validieren(): Boolean
Veranstaltung
-titel: String
-beschreibung: String
-dozent: Betreuer
+getTeilnehmer(): List<Student>
+anmelden(student: Student)

1. Nutzer wurde Abtrakte Klasse geändert, da Student, Betreuer und Verangestellter die gemeinsamen Eigenschaften besitzen

2. Rolle wurde beu Nutzer entfernt da wir die Vererbung nutzen werden

3. Die Klasse Observer wurde erstellt als Interface und enthält die Methode update()

4. Die Klasse Subject wurde erstellt als Interface und besitzt folgende Methoden: anfügen, trennen und Observer

Für Aufgabe 2 habe ich folgende 3 Entwurfmuster überlegt.

1. Factory Method für die Klassen Student, Betreuer, Verwaltungsangestellter.
folgende Änderungen sind dann zu tätigen:

Neues Interface NutzerFactory und eine Methode +createNutzer() dazu dann StudenFactory, BetreuerFactory und VerwaltungsFactory

2. Facade für DigitalerCampus eignet sich gut für eine Fassade und bündelt Pinwand, Nutzerverwaltung und LiveCampusKarte
<facade> DigitalterCampus dadurch spricht man nur mit DigitalerCampus z.b. campus.postErstellen()

3. Observer bei Post Kommentar Like und Nutzer wenn ein Post geliked oder kommentiert wird sollen Nutzer informiert werden

4. Buildern Pattern für Reservierung. Neue Klasse ReservierungBuilder mit folgenden Methoden: setRaum, setBesitzer, setStartZeit, setEndZeit, build die Methode für erstellen entfällt dadurch

Nutzer implements Observer
Post Implements Subject